ASSIGNMENT #1 – CLO-1
CAMPUS TASK BOARD
SOLVED REPORT

Fundamental React Concepts – Mini Project

Student Name: _____ Roll No.: _____

Semester / Session: _____ Date: _____

GitHub Repository: _____

Assignment Requirements

This solved submission follows the uploaded Assignment #1 instructions. The assignment requires a React-based Campus Task Board, at least three tasks, useState, a reusable TaskCard component, adding tasks without refreshing, useEffect output in the browser console, and Git/GitHub submission. ṆfileciteÖturn3file0ÖL20-L32Ọ

Task 1 – Create and Display Tasks (2 Marks)

Requirement: create at least three tasks, store them with useState, give each task an id, title and category, display them with map(), and assign a unique key. ṆfileciteÖturn3file0ÖL24-L32Ọ

```
// App.js
import { useState, useEffect } from "react";
import TaskCard from "./TaskCard";

function App() {
  const [tasks, setTasks] = useState([
    {
      id: 1,
      title: "Complete React Assignment",
      category: "Academic"
    },
    {
      id: 2,
      title: "Prepare Database Presentation",
      category: "University"
    },
    {
      id: 3,
      title: "Submit Project Report",
      category: "Project"
    }
  ]);

  return (
    <div>
      <h1>Campus Task Board</h1>

      {tasks.map((task) => (
        <TaskCard
          key={task.id}
          task={task}
        />
      ))}
    </div>
  );
```

```
}

export default App;
```

Explanation: tasks is an array stored in state. Each object contains id, title, and category. The map() method creates one TaskCard for every task, while key={task.id} provides React with a unique key.

Task 2 – Create a Reusable TaskCard Component (2 Marks)

The assignment requires a separate TaskCard component, passing task information through props, and displaying at least the task title and category. ṆfileciteÖturn3file0ÖL33-L38Ộ

```
// TaskCard.js
function TaskCard({ task }) {
  return (
    <div className="task-card">
      <h2>{task.title}</h2>
      <p>Category: {task.category}</p>
    </div>
  );
}

export default TaskCard;
```

Explanation: TaskCard is reusable because it does not contain fixed task data. It receives a task object through props and displays the title and category supplied by the parent component.

Task 3 – Add a New Task (2 Marks)

The assignment requires an input field, an Add Task button, an event handler, updating the list with useState, and displaying the new task without refreshing the page. ṆfileciteÖturn3file0ÖL39-L44Ộ

```
function App() {
  const [tasks, setTasks] = useState([
    { id: 1, title: "Complete React Assignment", category: "Academic" },
    { id: 2, title: "Prepare Database Presentation", category: "University" },
    { id: 3, title: "Submit Project Report", category: "Project" }
  ]);

  const [newTask, setNewTask] = useState("");

  const addTask = () => {
    const title = newTask.trim();

    if (title === "") {
      return;
    }

    const task = {
      id: Date.now(),
      title: title,
      category: "General"
    };

    setTasks((currentTasks) => [...currentTasks, task]);
    setNewTask("");
  };

  return (
    <div>
      <h1>Campus Task Board</h1>
```

```
      <input
        type="text"
        value={newTask}
        placeholder="Enter a new task"
        onChange={(e) => setNewTask(e.target.value)}
      />

      <button onClick={addTask}>Add Task</button>

      {tasks.map((task) => (
        <TaskCard key={task.id} task={task} />
      ))}
    </div>
  );
}
```

Result: when the user enters a task and clicks Add Task, a new object is appended to the state array. React immediately re-renders the list, so the new task appears without a page refresh.

Task 4 – useEffect (2 Marks)

The effect must run whenever the task list changes and print “Task list updated!” plus the total number of tasks in the browser console. ṆfileciteÖturn3file0ÖL45-L50Ộ

```
useEffect(() => {
  console.log("Task list updated!");
  console.log("Total number of tasks:", tasks.length);
}, [tasks]);
```

Explanation: [tasks] is the dependency array. Whenever setTasks() changes the tasks array, the effect executes. The console therefore reports the update and the current number of tasks.

Expected browser console output after the initial render:

```
Task list updated!
Total number of tasks: 3
```

After adding one new task:

```
Task list updated!
Total number of tasks: 4
```

The assignment specifically requires a screenshot of the browser console showing this useEffect output. ṆfileciteÖturn3file0ÖL14-L16Ộ

Complete App.js – Ready-to-Run Solution

```
import { useEffect, useState } from "react";
import TaskCard from "./TaskCard";
import "./App.css";

function App() {
  const [tasks, setTasks] = useState([
    {
      id: 1,
      title: "Complete React Assignment",
      category: "Academic"
    },
    {
      id: 2,
      title: "Prepare Database Presentation",
      category: "University"
```

```
    },
    {
      id: 3,
      title: "Submit Project Report",
      category: "Project"
    }
  ]);

  const [newTask, setNewTask] = useState("");

  useEffect(() => {
    console.log("Task list updated!");
    console.log("Total number of tasks:", tasks.length);
  }, [tasks]);

  const addTask = () => {
    const title = newTask.trim();

    if (title === "") {
      return;
    }

    const newTaskObject = {
      id: Date.now(),
      title: title,
      category: "General"
    };

    setTasks((currentTasks) => [
      ...currentTasks,
      newTaskObject
    ]);

    setNewTask("");
  };

  return (
    <div className="app">
      <h1>Campus Task Board</h1>

      <div className="task-input">
        <input
          type="text"
          value={newTask}
          placeholder="Enter a new task"
          onChange={(event) => setNewTask(event.target.value)}
        />

        <button onClick={addTask}>
          Add Task
        </button>
      </div>

      <div className="task-list">
        {tasks.map((task) => (
          <TaskCard
            key={task.id}
            task={task}
          />
        ))}
      </div>
    </div>
  );
}

export default App;
```

Complete TaskCard.js

```
function TaskCard({ task }) {
  return (
    <div className="task-card">
      <h2>{task.title}</h2>
      <p>Category: {task.category}</p>
    </div>
  );
}

export default TaskCard;
```

Optional App.css – Final Interface

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f4f6f8;
}

.app {
  max-width: 800px;
  margin: 50px auto;
  padding: 30px;
  background: white;
  border-radius: 12px;
}

.task-input {
  display: flex;
  gap: 10px;
  margin-bottom: 25px;
}

.task-input input {
  flex: 1;
  padding: 12px;
}

.task-input button {
  padding: 12px 20px;
  cursor: pointer;
}

.task-card {
  padding: 15px;
  margin-bottom: 12px;
  border: 1px solid #ddd;
  border-radius: 8px;
  background: #fafafa;
}

.task-card h2 {
  margin: 0 0 8px;
}
```

Task 5 – Git & GitHub Submission (2 Marks)

The assignment requires initializing Git, adding and committing project files, creating a GitHub repository, and pushing the project. The specified commands are listed in the assignment. ṆfileciteÖturn3file0ÖL51-L61Ộ

```
git init
git add .
```

```
git commit -m "Create Campus Task Board"
git branch -M main
git remote add origin YOUR_GITHUB_REPOSITORY_URL
git push -u origin main
```

Replace YOUR_GITHUB_REPOSITORY_URL with the URL of your own GitHub repository.

Project Structure

```
campus-task-board/
├── src/
│   ├── App.js
│   ├── App.css
│   ├── TaskCard.js
│   └── index.js
├── public/
├── package.json
└── README.md
```

How to Run the Project

```
npx create-react-app campus-task-board
cd campus-task-board

# Replace the generated src/App.js and create TaskCard.js
npm start
```

Open the local development address shown by React in the browser. Test the three initial tasks, enter a fourth task, click Add Task, and open the browser Developer Tools Console to verify the useEffect messages.

Expected Final Application

The completed interface should contain the heading “Campus Task Board”, an input field, an Add Task button, and at least three visible task cards. Each card should show its task title and category. Adding a new task should immediately create another card.

Example final tasks:

Task	Category
Complete React Assignment	Academic
Prepare Database Presentation	University
Submit Project Report	Project
Attend Web Engineering Lab	General

Submission Checklist

☐ GitHub repository link added.

☐ At least 3 initial tasks implemented with useState.

☐ Each task contains id, title and category.

☐ Tasks displayed using map().

☐ Unique key assigned using task.id.

☐ Separate reusable TaskCard component created.

☐ Task data passed to TaskCard through props.

☐ Input field and Add Task button implemented.

☐ New task appears without refreshing.

☐ useEffect depends on tasks.

☐ Console shows “Task list updated!” and total task count.

☐ Browser console screenshot captured.

☐ Final application screenshot captured.

☐ Complete source code included in submission.

☐ Git commits completed and project pushed to GitHub.

Conclusion

The Campus Task Board demonstrates the fundamental React concepts required by the assignment: state is managed with useState, task records are rendered with map(), TaskCard is a reusable component receiving props, new tasks are added through an event handler, and useEffect observes changes to the task list and reports the update in the browser console. Git and GitHub provide version control and submission of the completed project.